

Annexure A – Scenario Based Assignment

1. AI Drone for Emergency Medicine Delivery

i. Greedy Best-First Search

Greedy Best-First Search selects the path that appears **closest to the destination** using only the heuristic value $h(n)$.

Strengths:

- Very fast in many cases.
- Simple to implement.
- Useful when quick decisions are required.

Weaknesses:

- Does not consider the cost already travelled.
- May select a dangerous or costly path.
- Can give a non-optimal route when weather or terrain changes.

ii. A* Search

A* uses both the actual cost and heuristic:

$$f(n) = g(n) + h(n)$$

- $g(n)$ = cost already travelled.
- $h(n)$ = estimated remaining distance/cost.
- It balances **current cost and estimated future cost**.
- It can find an optimal path when the heuristic is admissible and costs are properly represented.

iii. Impact of Heuristic Accuracy

Algorithm	Effect of Accurate Heuristic	Effect of Poor Heuristic
Greedy Best-First	Quickly finds a good route	May choose a bad route
A*	Faster and more efficient	Explores more nodes and becomes slower

A good heuristic improves search speed and route quality.

iv. Effect of Dynamic Changes

If a road becomes blocked, the previously selected path may become invalid.

- Greedy search must search for another route.
- A* can update the path using the new costs.
- Frequent changes increase computation and delay.
- Real-time replanning is required for safety.

v. Recommendation

*A is the most suitable algorithm** because it considers both travel cost and estimated distance.

It provides:

- Better route quality.
- Consideration of terrain and weather costs.
- Better safety than Greedy Search.
- Ability to re-plan when paths become blocked.

For strict real-time operation, **dynamic/replanning A*** can be used.

2. Smart City Traffic Signal Optimization

Formulation as an Optimization Problem

The system can represent each traffic signal's timing as a decision variable.

Objective:

Minimize:

Total Cost = Waiting Time + Fuel Consumption + Traffic Congestion

Subject to:

- Signal timing limits.
- Pedestrian crossing requirements.
- Road capacity.
- Safety requirements.
- Minimum and maximum green-light time.

Why Traditional Search is Inefficient

Traditional search may need to examine a huge number of possible signal combinations.

With hundreds of intersections, the search space becomes extremely large. Traffic conditions also change continuously, making repeated exhaustive search impractical.

i. Hill Climbing

Hill Climbing starts with one signal-timing solution and changes it slightly to find a better solution.

Example:

- Current green time = 40 seconds.
- Change it to 45 seconds.
- If traffic improves, keep the change.
- Continue until no better nearby solution is found.

Limitations

- Can get stuck in a **local optimum**.
- May stop at a plateau.
- May have difficulty moving through ridges.
- Does not guarantee the global optimum.

ii. Local Maxima, Plateaus and Ridges

Local Maximum:

A solution is better than nearby solutions but not the best overall.

Example: One intersection improves, but the entire city could have a much better timing combination.

Plateau:

Many nearby solutions have almost the same performance.

Example: Changing a signal by a few seconds produces no noticeable improvement.

Ridge:

The best direction requires changing multiple variables together.

Example: Two connected traffic signals must both be changed to reduce congestion.

iii. Simulated Annealing and Genetic Algorithm

Simulated Annealing:

- Sometimes accepts a worse solution.

- This helps escape local optima.
- Gradually becomes more selective.
- Suitable for dynamic optimization.

Genetic Algorithm:

- Maintains a population of solutions.
- Uses selection, crossover and mutation.
- Can explore many possible solutions.
- Reduces the chance of getting trapped in one local optimum.

iv. Recommendation

Genetic Algorithm is a good choice for a large smart-city problem because it can explore many solutions simultaneously.

For continuous real-time adjustment, a **hybrid approach using Genetic Algorithm + local search** can provide better scalability and adaptability.

3. Autonomous Mars Rover

i. Why Online Search is Required

The rover does not have a complete map of Mars.

Therefore, it cannot calculate the complete path in advance.

An **online search agent**:

- Observes the environment.
- Takes an action.
- Receives new information.
- Updates its knowledge.
- Plans the next action.

This is suitable for unknown terrain.

ii. Challenges of Partial Observability and Dynamic Environment

Partial observability:

- Rover sensors see only limited areas.
- Some rocks, craters or hazards may be hidden.

- The rover has incomplete information.

Dynamic environment:

- New hazards may be discovered.
- Terrain conditions may change.
- Communication with Earth may be delayed.

Therefore, the rover must continuously adapt its decisions.

iii. Internal Model

The rover maintains an internal model containing:

- Known terrain.
- Safe and unsafe locations.
- Previously visited areas.
- Estimated distances.
- Possible hazards.
- Sensor observations.

When new sensor information is received, the model is updated and the rover replans.

iv. Online Search vs Other Strategies

Strategy	Information	Advantage	Limitation
Uninformed Search	No heuristic	Simple	Explores many states
Informed Search	Uses heuristic	Faster	Needs useful heuristic
Online Search	Learns while moving	Works in unknown environments	May need replanning

v. Recommendation

Online informed search is the most suitable approach.

It provides:

- Adaptability to unknown terrain.
- Real-time decision-making.

- Better use of limited sensor information.
 - Ability to avoid newly discovered hazards.
 - Safer navigation.
-

4. University Exam Timetable – CSP

Formulation as a Constraint Satisfaction Problem

A CSP consists of **Variables, Domains and Constraints**.

i. Variables, Domains and Constraints

Variables:

Each subject/exam is a variable.

Example:

- Mathematics
- Physics
- Computer Science
- English

Domains:

Possible time slots and examination halls.

Example:

Mathematics → Monday 9 AM, Monday 2 PM, Tuesday 9 AM

Constraints:

1. Two exams for the same student cannot overlap.
2. A hall cannot conduct two exams at the same time.
3. Some subjects must occur before others.
4. Faculty must be available.
5. Special accommodations must be provided.
6. Each exam must have an appropriate hall.

ii. Backtracking Search

Backtracking assigns a time and hall to each exam one by one.

Example:

1. Schedule Mathematics.
2. Schedule Physics.
3. Check constraints.
4. If a conflict occurs, undo the previous assignment.
5. Try another time slot.
6. Continue until a valid timetable is obtained.

iii. Constraint Propagation / Forward Checking

Forward checking removes invalid values from the domains of unassigned subjects.

Example:

If Mathematics is scheduled at Monday 9 AM, all exams that share students with Mathematics cannot use that time.

This:

- Reduces the search space.
- Detects conflicts early.
- Makes backtracking faster.

iv. Real-World Challenges and Best Strategy

Challenges include:

- Large number of subjects.
- Many students.
- Limited halls.
- Faculty availability.
- Special requirements.
- Multiple conflicting constraints.

Best strategy: Use **CSP with backtracking + constraint propagation + heuristics**.

Techniques such as **MRV (Minimum Remaining Values)** can select the most constrained exam first, improving scalability.

5. Strategic Game AI – Minimax and Alpha-Beta Pruning

i. Minimax Algorithm

Minimax models the game as a decision tree.

There are two players:

- **MAX:** AI player tries to maximize its score.
- **MIN:** Opponent tries to minimize AI's score.

The AI:

1. Generates possible moves.
2. Predicts opponent responses.
3. Evaluates future game states.
4. Selects the move with the best worst-case outcome.

ii. Computational Challenges

Game trees can become extremely large because every move can have many possible responses.

For example:

Current State → Many Moves → Many Opponent Moves → More Moves

Searching the complete tree can require a huge amount of:

- Processing time.
- Memory.
- Computation.

This is difficult in real-time games.

iii. Alpha-Beta Pruning

Alpha-Beta pruning removes branches that **cannot affect the final decision**.

- **Alpha (α):** Best value currently guaranteed for MAX.
- **Beta (β):** Best value currently guaranteed for MIN.
- When $\alpha \geq \beta$, remaining branches can be ignored.

Example:

If MAX already has a move guaranteeing a score of 8, and another branch can give MIN a result of 5 or less, MAX will never choose that branch. Therefore, the remaining unnecessary nodes can be pruned.

Benefit:

- Produces the same result as Minimax.
- Examines fewer nodes.
- Allows deeper search within the same time limit.

iv. Evaluation Function

A simple evaluation function can be:

Evaluation =

Material Advantage + Position Advantage + Resource Advantage + Safety + Mobility

Factors include:

- Number/strength of units.
- Control of important locations.
- Available resources.
- Defensive strength.
- Number of possible moves.
- Damage to opponent.

The weights can be adjusted according to the game.

v. Recommendation

For real-time strategy games, use:

Depth-Limited Minimax + Alpha-Beta Pruning + Heuristic Evaluation

This approach:

- Gives strong decisions.
- Reduces unnecessary computation.
- Works within time limits.
- Allows deeper searches.
- Provides a good balance between **optimality and real-time performance**.